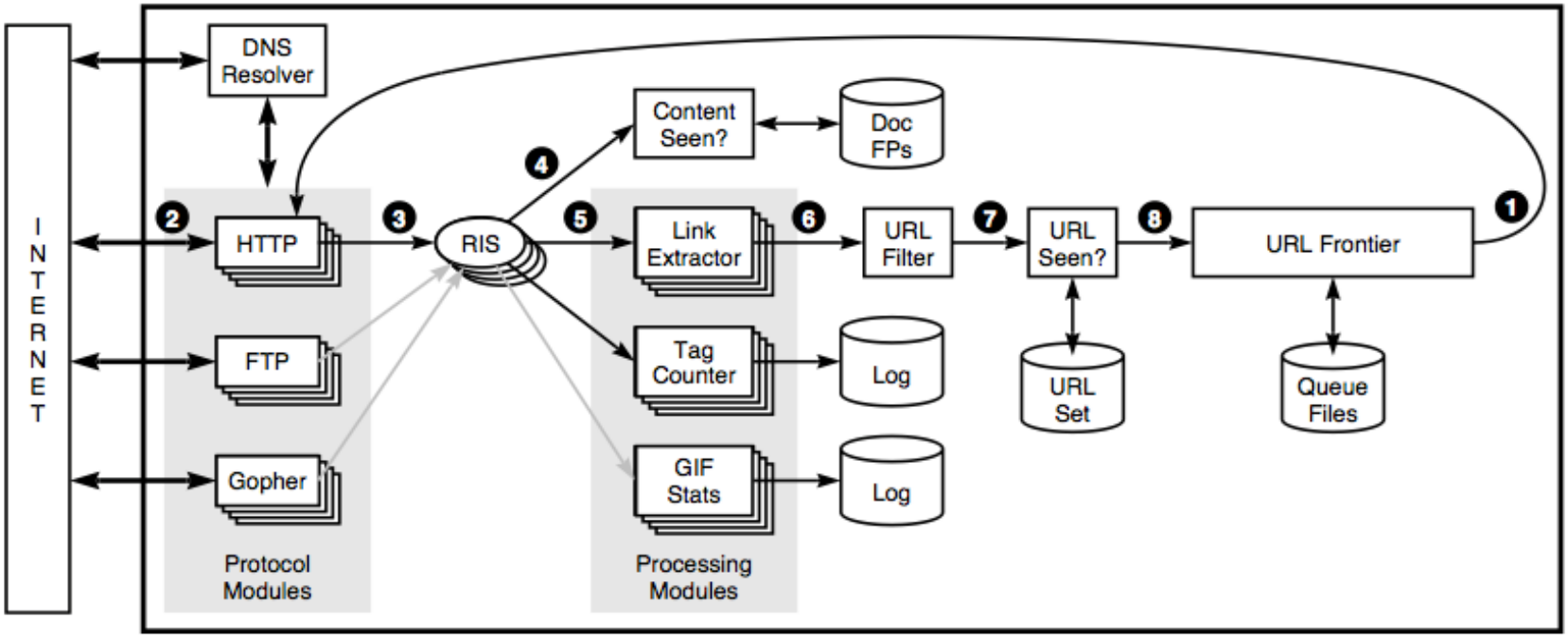
Scaling Web Search

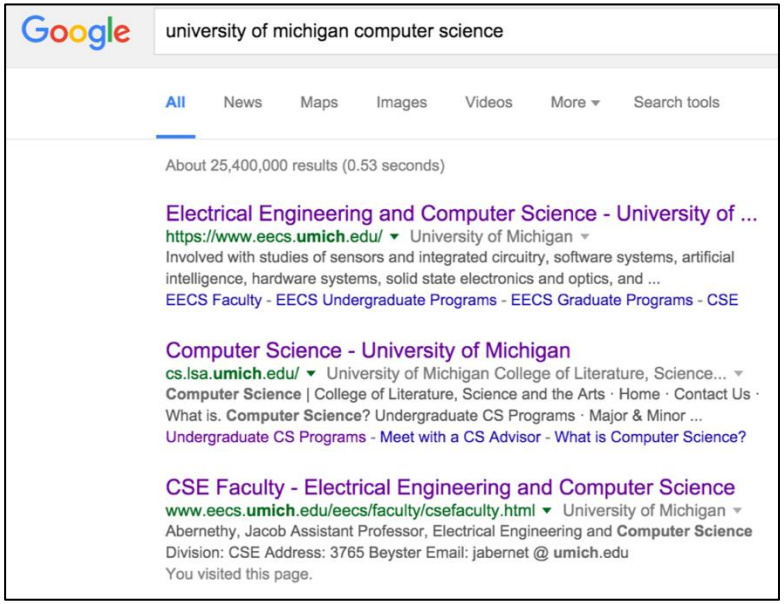


Review: How search works

- Ahead of time
  - Download every web page
  - Build an index
- Just-in-time (query time)
  - Read index
  - Rank results

Review: Ranking results

- Problem: How do we rank results?
- ***Text analysis****:* Rank using words on page
- ***Link analysis****:* Rank using links on page



Where we are

- How do we download every web page?
- How do we build an index?
- How do we make it fast?

A few numbers

- 5 B - 100 B web pages
  - Let’s say 10 Billion for concreteness
- Assume 10 KB per compressed page
  - 100 TB data to index
- 1 minute - 1 month freshness
- 3-5 B queries *per day* on Google alone

Agenda

- **Crawlers**
- Deduplication
- Inverted index construction
- Distributed search architecture

Crawlers

- To search the web, we need to build an index and link graphs

- Problem: We need a copy of the entire web
- Solution: Download the entire web

- Web Robots AKA Web Wanderers, Crawlers, or Spiders
- Example: `googlebot`
  - http://www.robotstxt.org/db/googlebot.html

- Problem: no list of all web pages

Web crawling

- Problem: No list of all web pages
- Solution: Traversal of the link graph

Web crawling: graph traversal

- Have a list of "seed pages", follow all links like a BFS/DFS
- Most of the Internet is a connected component

Problem1:Password protected content

- Problem: Password-protected content
  - Bank records, Facebook, SnapChat, Dropbox, Google products, there's a lot
  - Crawler can't download it

- No solution. Can't index these pages.

Problem2: Deep web

- *Surface web*: content indexed by search engines
- *Deep web*: content not indexed by search engines
- Size of deep web likely several orders of magnitude larger than surface web

SURFACE WEB

Google

Bing

Wikipedia

DEEP WEB

Academic Information

Medical Records

Legal Documents

Scientific Reports

Subscription Information

Contains 90% of the information on the Internet, but is not accessible by Surface Web crawlers.

Social Media

Multilingual Databases

Financial Records

Government Resources

Competitor Websites

Organization-specific Repositories

(DARK WEB)

A part of the Deep Web accessible only through certain browsers such as Tor designed to ensure anonymity. ***Deep Web Technologies has zero involvement with the Dark Web.***

Illegal Information

Drug Trafficking sites

TOR-Encrypted sites

Political Protests

Private Communications

Problem3: Client-side dynamic pages

- Problem: How to download client-side dynamic pages?
- Solution 1: Skip those pages
- Solution 2: JS interpreter in crawler

```
<html>
<body>
   <div id="reactEntry>Loading...</div>
   <script src="..."></script>
</body>
</html>
```

Problem4: Crawlers "not nice" to servers

- Problem: Crawlers generate a **lot** of traffic
  - CPU and bandwidth costs money

- Website owners don't like if you crawl too fast and will block you

- Solution: Limit # requests per site per second

Problem5: Not all sites want to be indexed

- Problem: Some websites don't want to be indexed

- Reasons not to index:
  - Personal creations: e.g. artwork on Google Images
  - Parts of dynamic websites: e.g. search engines

- Solution: `robots.txt`

robots.txt

https://github.com/robots.txt

```
User-agent: Googlebot
Disallow: /*/download
Disallow: /*/revisions
Disallow: /*/*/issues/new
Disallow: /*/*/issues/search
Disallow: /*/*/commits/*/*
Disallow: /*/*/commits/*?author
Disallow: /*/*/commits/*?path
...
```

Problem 6: Deduplication

- Many web pages are exact copies of other web pages
  - Only create index once

- We'll learn how to solve this in the next segments

Agenda

- Crawlers
- **Deduplication**
- Inverted index construction
- Distributed search architecture

Deduplication in Web Search

- Many web pages are identical or nearly identical
  - News websites: many have the same articles

- Storing duplicates wastes space and slow search
- Goal: detect and remove duplicate or near-duplicate pages efficiently

- Key idea: avoid storing or ranking the same content multiple times.

AP

Top Stories Topics Vide

Pistons hire EMU coach to be G League team's GM

yesterday

DETROIT (AP) — Rob Murphy has been hired as pres
Detroit Pistons' G League affiliate.

The Pistons announced the move Wednesday. Murphy
Eastern Michigan for the past 10 seasons.

Menu The Seattle Times Sports Log In | Subscribe | Search

CORONAVIRUS LOCAL BIZ SPORTS ENTERTAINMENT LIFE HOMES OPINION | JOBS EXPLORE All Sectio

Seahawks M's UW WSU Storm Sounders Kraken Preps Stone Calkins Wilner TV/Radio

Sports

Pistons hire EMU coach to be G League team's GM

March 17, 2021 at 8:58 am

By The Associated Press
The Associated Press

DETROIT (AP) — Rob Murphy has been hired as president and general manager of the Detroit Pistons' G League affiliate.

Menu WXYZ DETROIT

Lions Tigers Red Wings Pistons College Sports High School Sports Golf Detroit Grand Prix

SPORTS

Pistons hire Eastern Michigan coach Rob Murphy to be G League team's GM

Exact Duplicate Detection (Hashing)

- Compute a hash for each document
  - Hash("It grows in dry places...") = 8d2f908...
- If two documents have the same hash; they are identical
- Very fast and scalable
- Ex: hash(doc1) == hash(doc2) => duplicate
- With small doc change; completely different hash

Limitation of Hashing

- Every small edits change the hash completely different
- Example:
  - “best pizza in town”
  - “best pizza in town!”
- These are nearly identical, but hashes differ
- Need away to detect near duplicates, not just exact matches

Why not compare entire document?

- Documents can be similar but not exactly the same

AP

What's for dinner? Depends what's in the fridge and pantry

RELATED TOPICS

Lifestyle

General News

Cooking

Just about now, you might be tired of cooking and eating what's in your refrigerator. We are all used to asking what's for dinner? What are we in the mood for — pizza, sushi, Mexican? And then going to a restaurant to get it.

Now, with the coronavirus forcing many of us to stay home more, it's time to look at mealtime and cooking at home a little differently. Instead of asking, `What I am in the mood for?' I am

ngredients dictate what I

xed unsalted nuts because

of "anti-oxidants" a day for

p my favorite sweet and

gar and Spice Candied

ious and welcome nibble

abcNEWS

LOG IN

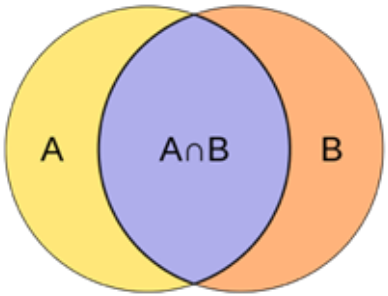
Just about now, you might be tired of cooking and eating what's in your refrigerator. We are all used to asking what's for dinner? What are we in the mood for — pizza, sushi, Mexican? And then going to a restaurant to get it.

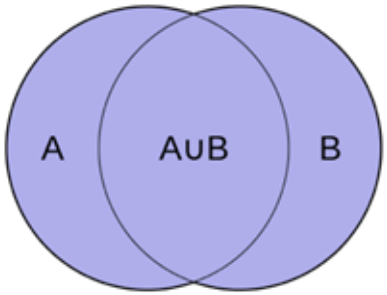
Now, with the coronavirus forcing many of us to stay home more, it's time to look at mealtime and cooking at home a little differently. Instead of asking, `What I am in the mood for?' I am looking more closely in my pantry and my refrigerator and letting my ingredients dictate what I make.

Measuring Similarity with Jaccard

- Represent each document as a set of words
- Compare overlap between sets

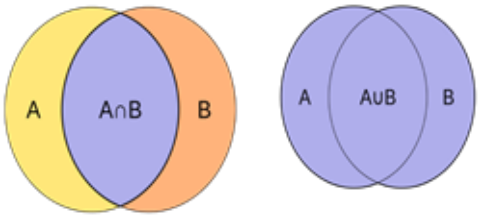
$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}.$$





- 1 = identical; 0 = completely different
- Measures how similar two document are

Jaccard Example



$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}.$$

- Doc A = {cooking, dinner, recipe}
- Doc B = {cooking, dinner, meal}

- Intersection = {cooking, dinner} -> 2
- Union = {cooking, dinner, recipe, meal} -> 4

- Similarity = J(A, B) = 2/4 = 0.5

- Higher overlap -> higher similarity

Why this Doesn't Scale

- Comparing all pairs is too expensive
- For N documents → ~ N^2 comparisons
- Web-scale data makes this infeasible

- Need a faster approximation method

String comparison

- Problem: Comparing strings is slow, even short strings

- A = {cooking, dinner, recipe}
- B = {cooking, dinner, meal}
- J(A, B) = 2/4 = 1/2

Numeric comparison

Word	Hash
cooking	0x1a4
dinner	0x2b5
recipe	0x3c6
meal	0x4d7

- Problem: Comparing strings is slow, even short strings

- A = {cooking, dinner, recipe}
- B = {cooking, dinner, meal}
- J(A, B) = 2/4 = 1/2

- Solution: Hash each word and compare the numbers

- A_hashes = {0x1a4, 0x2b5, 0x3c6}
- B_hashes = {0x1a4, 0x2b5, 0x4d7}
- J(A_hashes, B_hashes) = 2/4 = 1/2

Sampling

Word	Hash
cooking	0x1a4
dinner	0x2b5
recipe	0x3c6
meal	0x4d7

- Problem: Set comparison is still too slow
- Solution: Compare a **sample** of the set

- Randomly select k values from A and k values from B
- Example k=2
  - A_hashes = {**0x1a4**, **0x2b5**, ~~0x3c6, ...~~}
  - B_hashes = {**0x1a4**, **0x2b5**, ~~0x4d7, ...~~}

- A_sample = {0x1a4, 0x2b5}
- B_sample = {0x1a4, 0x2b5}
- J(A_sample, B_sample) = 2/2 = 1

Hashing and random values

- Problem: How to select a random hash value?

- Solution 1: Select the element at a random index
  - What if container doesn't support random access?
  - Must store all the values first

- Solution 2: Select the min hashed value
  - The hash function maps inputs uniformly over the output
  - Selecting the *min(h(x))* is the same as selecting a random item x!
  - This is called **MinHash**

MinHash

- Use multiple hash functions
- For each document:
  - Hash all elements
  - Keep only the minimum hash value
- Repeat with different hash functions
- Each document -> small signature

MinHash example

Word	Hash
cooking	0x1a4
dinner	0x2b5
recipe	0x3c6
meal	0x4d7

- Doc A = “cooking dinner recipe”
- Doc B = “cooking dinner meal”

- MinHash with k=2
- A_sample = {0x1a4, 0x2b5}
- B_sample = {0x1a4, 0x2b5}

- Key idea: Compress large sets into small comparable signatures.

Why MinHash works

- Probability that two documents have same min hash = their Jaccard similarity

- So:
  - More matches -> more similar
  - Fewer matches -> less similar

- MinHash approximates similarity without full comparison

Shingling

- Break document into sequences of words (k-grams)

- Example (k=2):
  - “web search engine” -> {web search, search engine”}

- Captures word order
- Improve similarity accuracy

- Preserves structure, not just word presence

MinHash with shingles

Shingle	Hash
this is	0x1aa
is some	0x2bb
some text	0x3cc

- Apply MinHash to **shingles** instead of words

- Example
  - doc = "this is some text”
  - 2-shingles
  - k=2

- Shingles = {0x1aa, 0x2bb}

MinHash with shingles advantages

- Comparing numbers is faster
  - Compare hash values (numbers) instead of strings
- Approximation is faster
  - Compare k shingles instead of n
- Fast random shingle selection
  - Select k shingles using minimum hash value

Agenda

- Crawlers
- Deduplication
- **Inverted index construction**
- Distributed search architecture

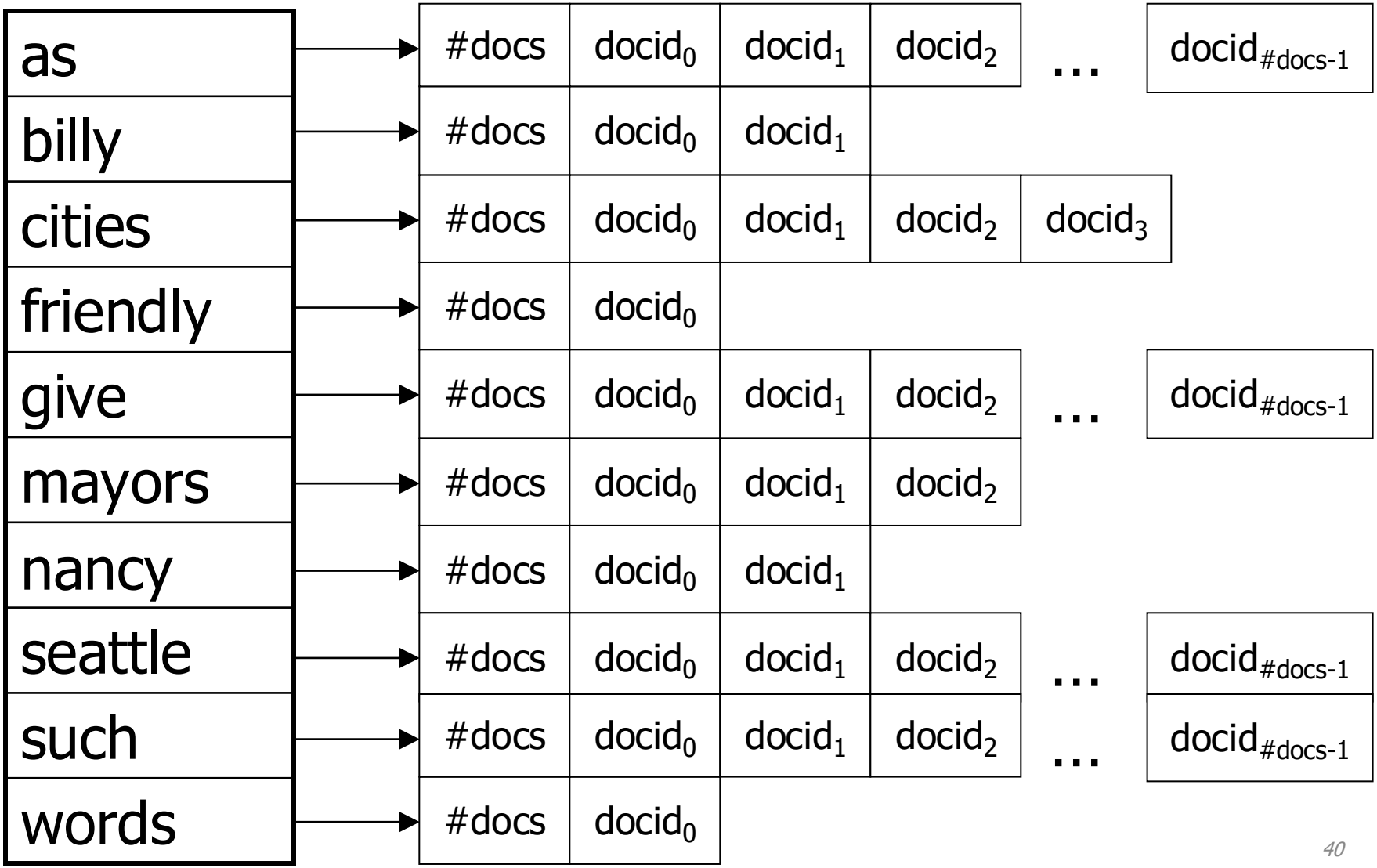
Serving search results

- After crawling is finished, we have a big database of documents
- Now we need to build an index

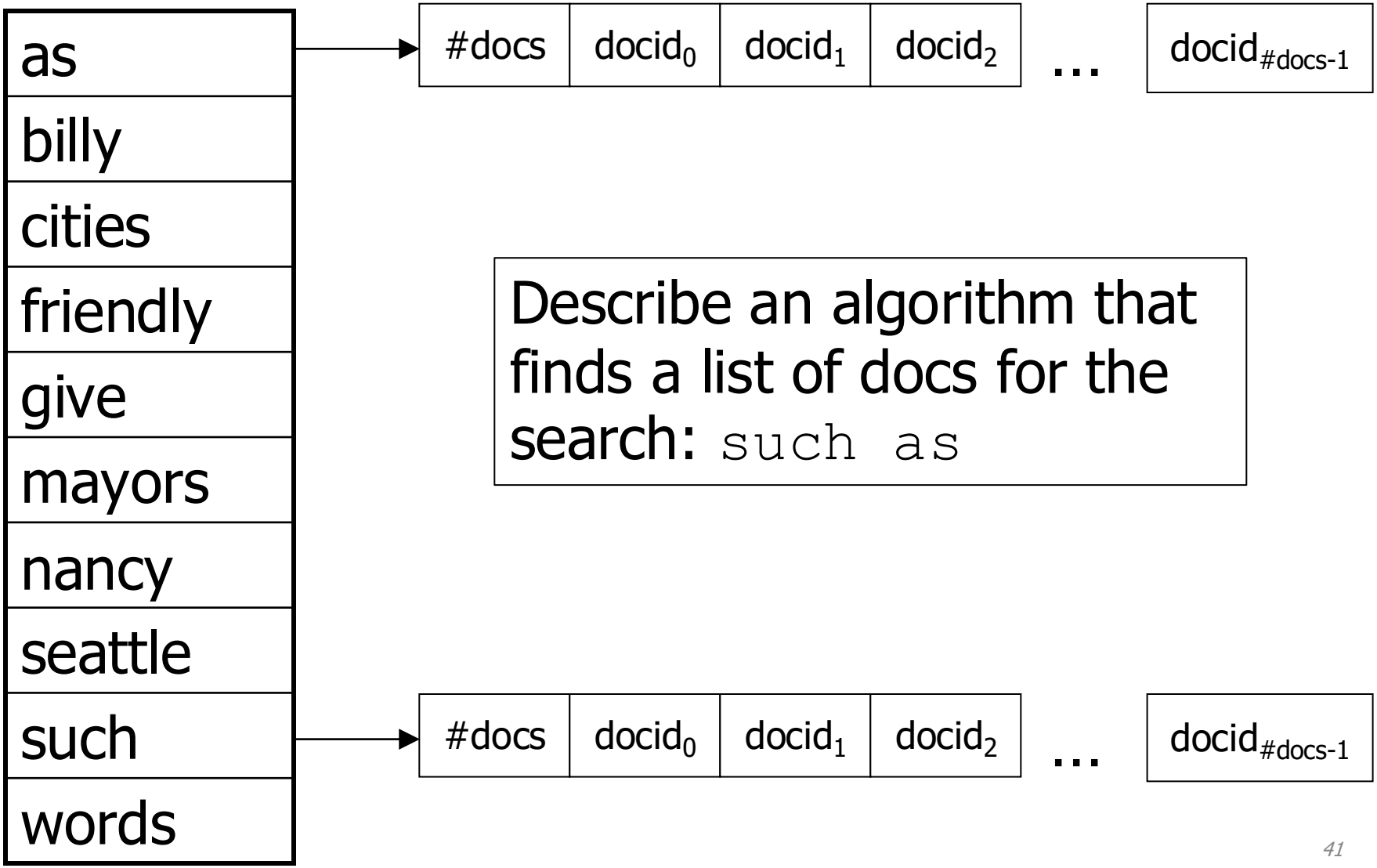
Inverted index

- A ***forward index*** is a list of words in each document
  - Doc -> words
- An ***inverted index*** maps words to docs that contain those words
  - Word -> docs
- For each word, list all the documents where that word can be found
- Key to fast query processing

Inverted index



Inverted index exercise



Describe an algorithm that finds a list of docs for the search: `such as`

Solution: merge intersection

as
billy
cities
friendly
give
mayors
nancy
seattle
such
words

#docs	docid$_0$	docid$_1$	docid$_2$	…	docid$_{\#docs-1}$
104	21	150	322		2501

1. Test for equality
2. Advance smaller pointer
3. Abort when a list is exhausted

Returned docs: 322

#docs	docid$_0$	docid$_1$	docid$_2$	…	docid$_{\#docs-1}$
15	99	322	426		1309

Agenda

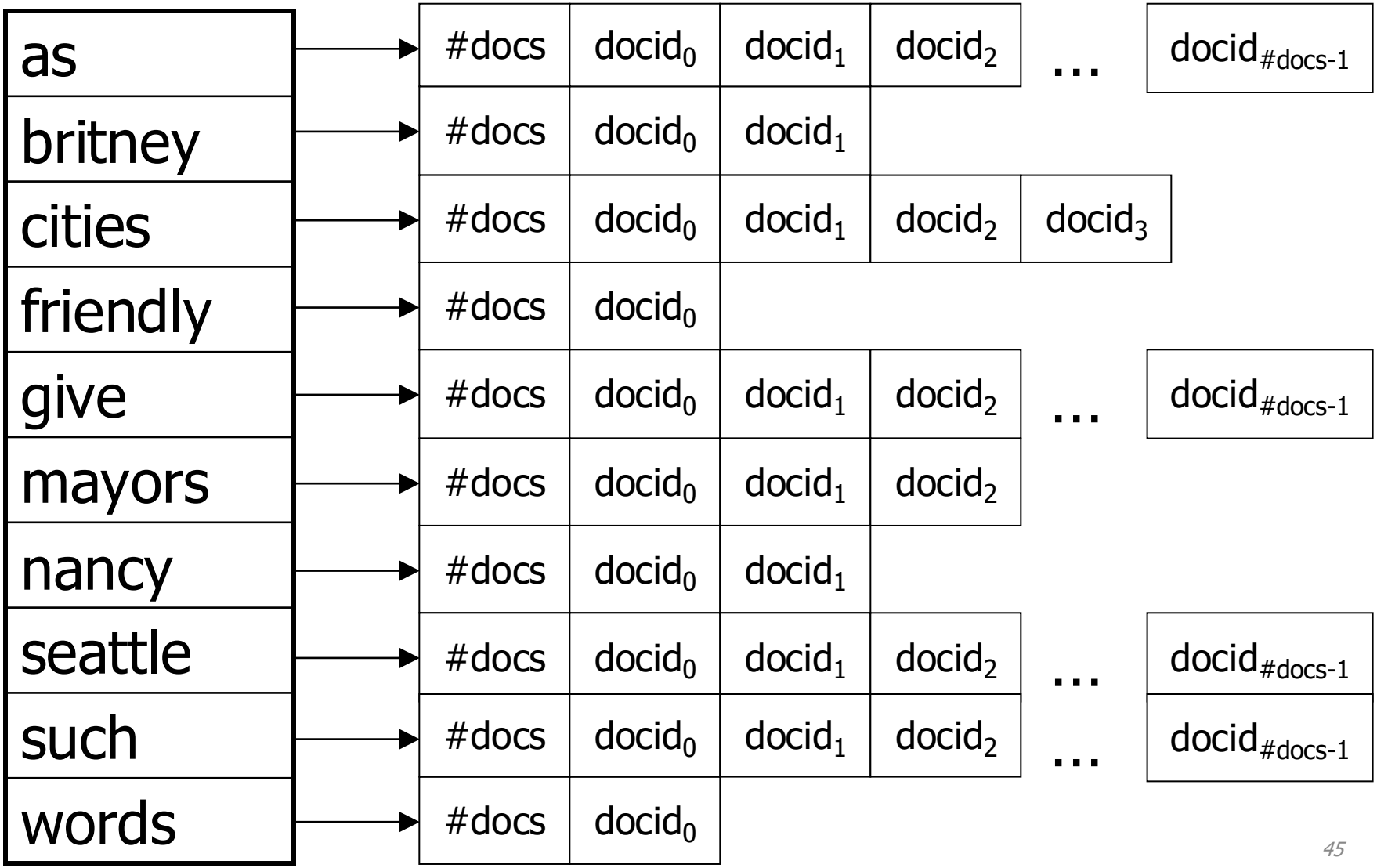
- Crawlers
- Inverted index construction
- Deduplication
- **Distributed search architecture**

Distributed search

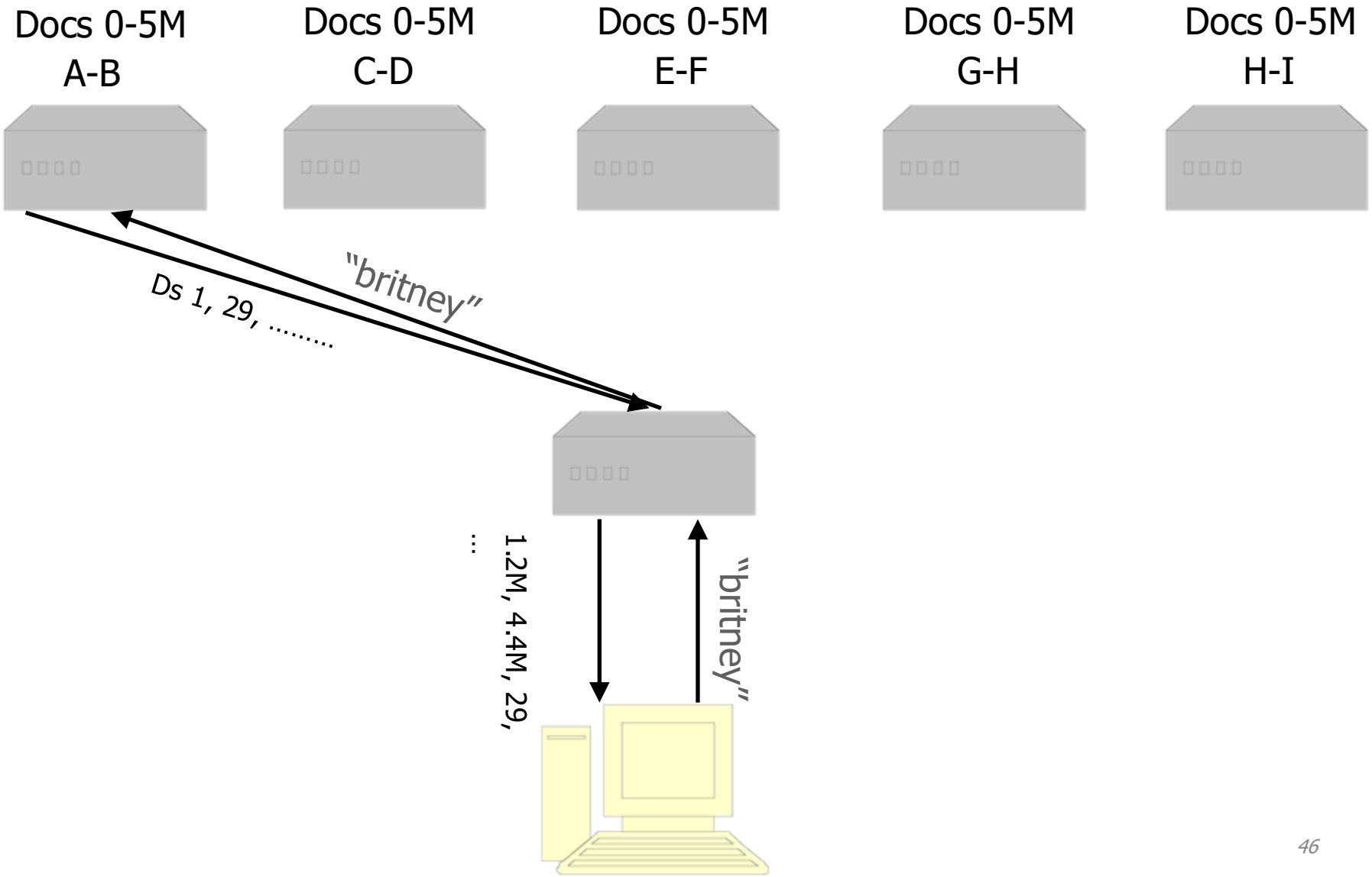
- Problem: Inverted index is too big for one machine
  - Billions of docs
- Problem: Query load is too big for one machine
  - Billions queries
- Problem: A machine could fail

- Solution: Parallel query processing
  - Segment by term
  - Segment by document

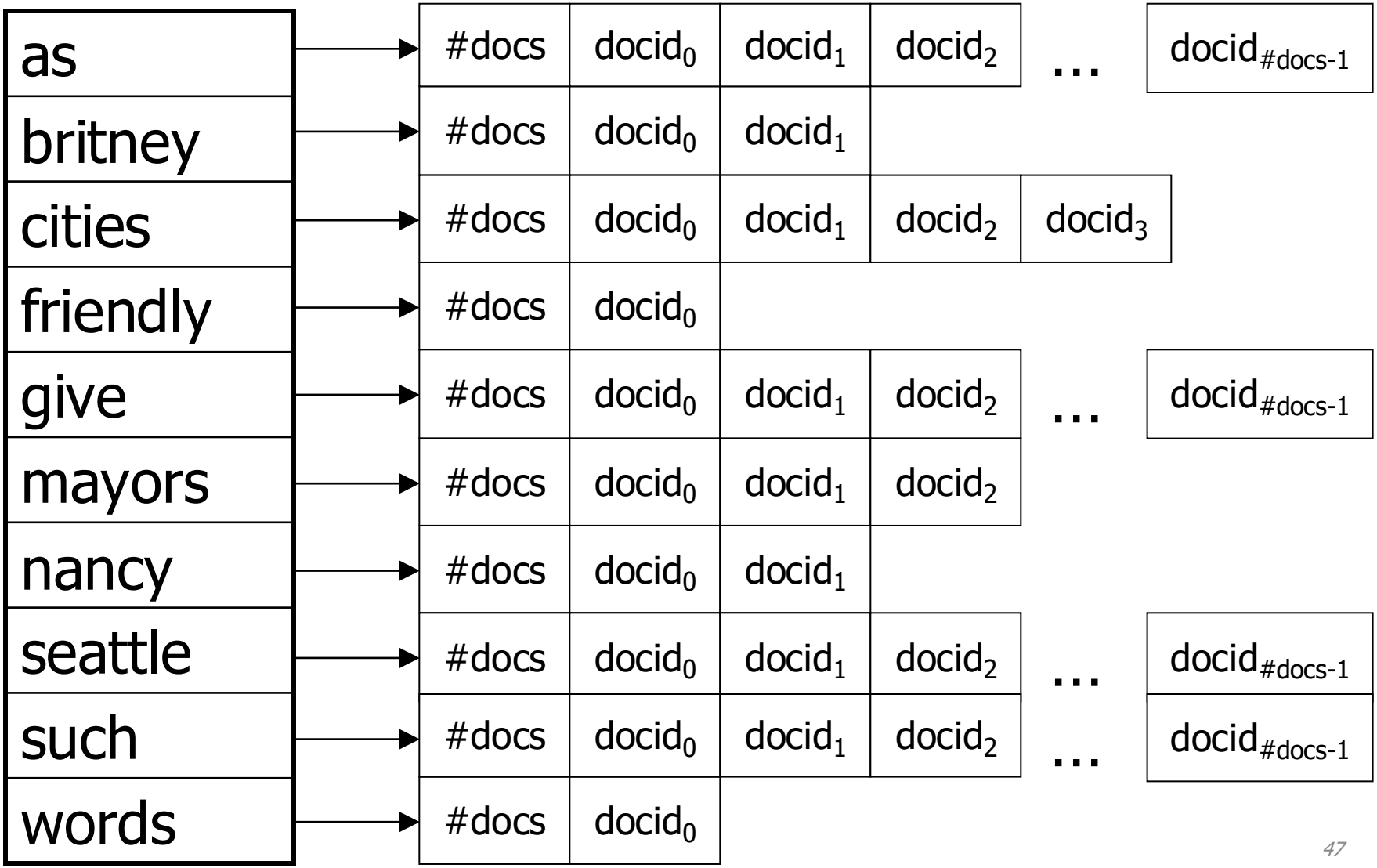
Segment by term (divide rows)



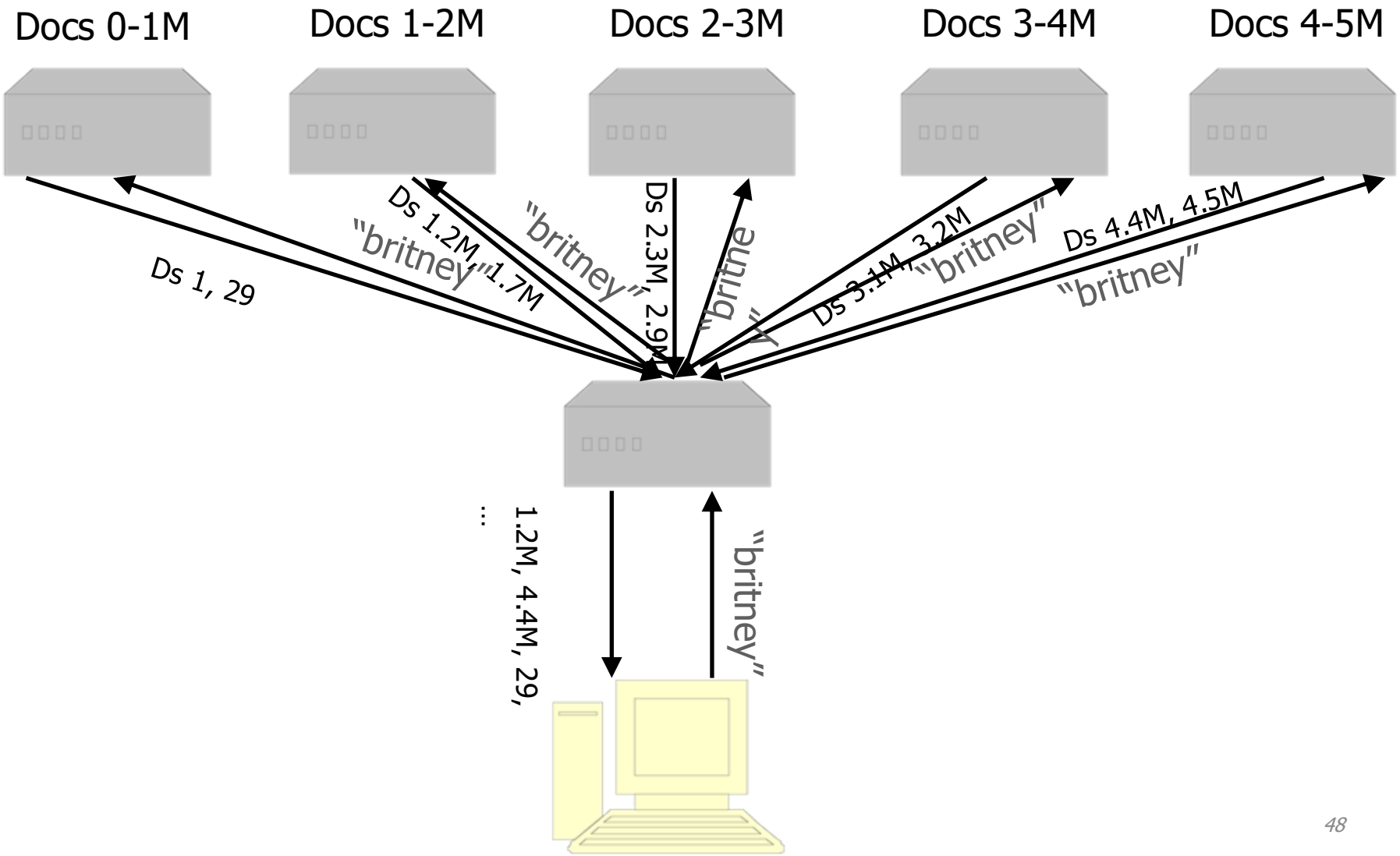
Segment by term



Segment by document (divide cols)



Segment by document



Segmentation

- What are the tradeoffs of segment-by-term vs. segment-by-document?
- What happens if a machine dies?

Segmentation

- Segment by document
  - Easy to partition (docid % num_segments)
  - Easy to add new documents
  - If a machine fails, quality decreases, but queries don’t die
- Segment by term
  - Harder to partition (terms uneven)
  - Trickier to add a new document (need to touch many machines)
  - If machine fails, search term might disappear, but not critical pages (e.g., cnn.com/index.html)

Summary: Crawlers

- Problem: Need to build index and link graphs
- Solution: Download the web with a crawler

- Problem: No list of all web pages
- Solution: Traversal of the link graph

- Surface web: content indexed by search engines

Summary: Deduplication

- Deduplication: filter out duplicate web pages
- Shingle: sequence of words
- Hash of shingle: numeric representation of sequence of words
- Min hash: random shingle selection
- Compare 2 set-of-random-hashes with Jaccard similarity coefficient

Summary: Inverted Index

- Inverted index: map words to docs that contain words

- Problem: Inverted index is too big
- Problem: Query load is too big
- Solution: Parallel query processing

- Segment by term: different terms on different machines
- Segment by document: different documents on different machines